

Meta-heurística: GRASP

Kaíque M. Lima

Universidade Tecnológica Federal do Paraná (UTFPR)

Santa Helena, PR – Brasil

Pesquisa e Ordenação – Prof. Glória Patrícia Lopez Sepulveda

kaiquelima@alunos.utfpr.edu.br

Abstract

a

Resumo

b

1 Introdução

Problemas de otimização combinatória complexos (tipicamente NP-difíceis) são desafiadores de resolver exatamente em tempo hábil. Assim, recorre-se a heurísticas e meta-heurísticas para obter soluções aproximadas de boa qualidade em prazos razoáveis. Meta-heurísticas são estratégias de busca de alto nível que guiam heurísticas básicas para explorar eficientemente o espaço de soluções e evitar ótimos locais. Dentre as meta-heurísticas bem-sucedidas na literatura destacam-se Simulated Annealing, Busca Tabu, Algoritmos Genéticos, Iterated Local Search, Variable Neighborhood Search e o GRASP (*Greedy Randomized Adaptive Search Procedure*).

O GRASP foi introduzido por Feo e Resende em meados dos anos 1990 como um procedimento iterativo multi-início para problemas de otimização combinatória. Cada iteração do GRASP consiste em duas fases: uma fase de construção, na qual é construída uma solução viável (inicial) de forma gulosa e aleatória, e uma fase de busca local, na qual essa solução é iterativamente melhorada buscando um ótimo local em sua vizinhança. Ao final, o melhor resultado encontrado entre todas as iterações é retornado como solução. Uma característica atraente do GRASP é a sua simplicidade de implementação e o reduzido número de parâmetros a calibrar, permitindo que o esforço do desenvolvedor concentre-se em estruturas de dados eficientes para agilizar as iterações. De fato, o GRASP é reconhecido pela habilidade de encontrar soluções de alta qualidade com relativa facilidade de ajuste, sendo aplicado com sucesso em uma grande variedade de problemas práticos.

Este artigo está organizado da seguinte forma: na Seção seguinte, descreve-se em detalhes o funcionamento do algoritmo GRASP. Em seguida, são apresentadas algumas áreas de aplicação típicas do GRASP na literatura. Na quarta seção, um estudo de caso real é explorado, demonstrando a aplicação da meta-heurística em um problema concreto e comparando seu desempenho com uma abordagem gulosa tradicional. Por fim, são tecidas conclusões sobre a eficácia do GRASP e sugestões de trabalhos futuros.

2 Descrição da atividade

2.1 O Algoritmo GRASP

O GRASP é uma meta-heurística construtiva e iterativa composta por duas etapas principais em cada iteração: construção e busca local. O Algoritmo 1 resume de forma simplificada o procedimento do GRASP:

Algorithm 1: GRASP

```
Result:  $S_{\text{best}}$ 
 $S_{\text{best}} \leftarrow \infty$ 
while condição de parada não satisfeita do
     $S \leftarrow \text{ConstruçãoGulosaRandomizada}()$ 
     $S \leftarrow \text{BuscaLocal}(S)$ 
    if  $f(S) < f(S_{\text{best}})$  then
         $S_{\text{best}} \leftarrow S$ 
return  $S_{\text{best}}$ 
```

Algoritmo 1: Pseudocódigo simplificado do procedimento GRASP, onde S_{best} denota a melhor solução encontrada até o momento (minimizando f).

2.2 Fase de Construção

Inicia com uma solução parcial vazia e insere gradualmente elementos até obter uma solução completa viável. Em cada passo da construção, todos os candidatos a serem inseridos são avaliados por uma função de avaliação gulosa, que estima o benefício (ou custo) de incluir cada elemento na solução parcial. Essa função é adaptativa, pois sua avaliação dos candidatos é recalculada a cada inserção, refletindo as mudanças na solução parcial. A partir dessa avaliação, determina-se um subconjunto dos melhores candidatos – a chamada Lista Restrita de Candidatos (RCL) – contendo os elementos de melhor valor segundo o critério guloso. Diferentemente de um algoritmo puramente guloso, que escolheria sempre o melhor candidato, o GRASP seleciona aleatoriamente um dos elementos da RCL para inserir na solução. Esse processo híbrido (guloso + aleatório) confere diversificação à busca e faz com que o método seja classificado como semi-guloso. O tamanho ou critério de inclusão da RCL pode ser controlado por um parâmetro α que define o nível de aleatoriedade: $\alpha=0$ equivale a uma escolha totalmente gulosa (RCL contendo apenas o melhor elemento), enquanto $\alpha=1$ permitiria escolhas aleatórias amplas; valores intermediários equilibram os dois extremos. Ao final da fase de construção, obtém-se uma solução viável S para o problema.

2.3 Fase de Construção

Dada a solução S construída, aplica-se uma busca local em sua vizinhança para melhorar a qualidade da solução (minimizando a função objetivo). A busca local consiste em aplicar sucessivas modificações na solução (por exemplo, trocas, realocações ou outras operações dependendo do problema) para encontrar uma solução vizinha de menor custo. Essas operações continuam até que se atinja um ótimo local, ou seja, até que nenhuma melhoria adicional seja possível mediante as mudanças definidas. A solução resultante S' (ótimo local) é então comparada com a melhor solução global encontrada até o momento;

se S' for melhor, passa a ser a nova melhor solução S_{best} . O algoritmo então inicia uma nova iteração (nova construção seguida de busca local), repetindo o processo até satisfazer algum critério de parada (por exemplo, número máximo de iterações, tempo de CPU limite ou convergência).

2.4 Parâmetros e Variações

O GRASP tipicamente requer a definição do número de iterações (ou do critério de parada) e do parâmetro β que controla a aleatoriedade/greediness na seleção da RCL. Uma vantagem importante é que o método normalmente não é muito sensível a pequenos ajustes de parâmetros, e poucos parâmetros precisam ser ajustados manualmente. Além da versão básica, diversas variantes do GRASP foram propostas. Por exemplo, o GRASP Reativo ajusta dinamicamente o valor de β durante as iterações, de acordo com a qualidade das soluções obtidas, buscando equilibrar adaptativamente intensificação e diversificação. Outras extensões incluem o uso de técnicas de path-relinking (reconexão de caminhos) pós-busca local para combinar soluções elite e explorar melhor o espaço, além de implementações em paralelo para aproveitar múltiplos processadores. Em todos os casos, o princípio fundamental de construir iterativamente soluções semi-aleatórias de qualidade e refiná-las via busca local permanece como núcleo do GRASP.

3 Estudo de Caso Real

Para ilustrar o desempenho do GRASP em um contexto prático, considera-se um **problema real de logística de distribuição**, conhecido como **Problema de Localização de Hubs Capacitados (PpHC)**. Nesse problema, deseja-se selecionar locais (hubs) para instalação de centros de distribuição e atribuir clientes a esses hubs, de forma a minimizar o custo total de transporte (por exemplo, a soma das distâncias percorridas), respeitando a capacidade de atendimento de cada hub. Trata-se de um problema NP-difícil, típico em planejamento logístico. Neste estudo de caso, foi implementado um algoritmo GRASP para resolver instâncias do PpHC baseadas em dados reais de uma empresa de entregas, comparando-se os resultados com um algoritmo puramente **guloso** equivalente. O algoritmo guloso constrói a solução atribuindo cada cliente ao hub mais próximo disponível, enquanto o GRASP realiza múltiplas construções aleatorizadas e busca local para refinar as soluções.

Foram testadas 12 instâncias de diferentes tamanhos, variando de 50 até 200 clientes, em três regiões distintas (por exemplo, conjuntos de cidades no Distrito Federal, Rio de Janeiro, etc.). A ****Tabela 1**** resume os resultados de custo obtidos pelo algoritmo guloso vs. o GRASP em uma das instâncias (região do Rio de Janeiro, RJ), para variados números de clientes:

Tabela 1 – Comparativo de custo total da solução (distância) obtido pelo Algoritmo Guloso versus pelo GRASP (instâncias da região RJ).

[illegible]

Observa-se que o GRASP encontrou soluções de custo ****drasticamente menor**** que o heurístico guloso em todas as configurações. Por exemplo, para a instância com 50 clientes, o custo total obtido pelo método guloso foi 48.9, enquanto o GRASP produziu uma solução de custo 4.0 – uma redução de mais de 90%. De modo similar, na instância

com 200 clientes, o custo caiu de 506.9 (guloso) para 31.4 com o GRASP. Essa enorme diferença indica que o algoritmo guloso provavelmente ficou preso em uma solução muito subótima (por exemplo, utilizando poucos hubs e acarretando longas distâncias de transporte), ao passo que o GRASP explorou diversas configurações e encontrou uma solução muito mais eficiente.

Além da qualidade das soluções, é importante notar o **tempo de computação** dos métodos. Embora o algoritmo guloso seja extremamente rápido (por construir a solução em uma única passada), o GRASP também manteve tempos de execução baixos – da ordem de décimos de segundo nessas instâncias. No caso acima, o guloso levou cerca de 0,05 segundos na instância de 200 clientes, enquanto o GRASP consumiu aproximadamente 0,24 segundos. Ou seja, mesmo executando múltiplas iterações e incorporando busca local, o GRASP alcançou **tempos computacionais similares** aos do guloso nas instâncias testadas. Em todos os 12 cenários avaliados, o GRASP produziu soluções de menor custo que o algoritmo guloso, confirmando a viabilidade do uso dessa meta-heurística para melhorar a logística de distribuição na prática. Esses resultados reforçam que estratégias gulosas puras, embora rápidas, podem falhar em encontrar boas soluções em problemas complexos, ao passo que meta-heurísticas como o GRASP oferecem um equilíbrio promissor entre qualidade da solução e custo computacional.

4 Conclusão

Referências